

Tech Round Assignment — AI Agent Developer

Overview

Design and build a **working agentic solution** for a financial data assistant.

The system ingests a large volume of company financial data (Excel + PDF), turns it into something an agent can reason over, enforces **role-based access control**, and improves over time through a **feedback loop**. A user should be able to ask questions in natural language, and the agents should fetch the right data, respect access rules, and carry out the next steps.

The guiding principle for this assignment: **minimal fluff** — the solution should actually work. A smaller system that runs end-to-end beats an elaborate design that doesn't.

Timeline & Communication

You have **24 hours** from the time you receive this brief to build, submit, and share a walkthrough.

Questions are encouraged — but there's a window. Email us your questions within **1 hour** of receiving this assignment. Treat scoping and clarifying questions as part of the exercise — asking good questions is a positive signal, not a negative one. Get them in early.

We'll reply to questions raised inside that first hour as quickly as we can. Questions sent after the window may not get a response before your submission is due.

What to Build

1. Data ingestion

Take real financial data from **any public source** — e.g. Apple's last 3 years of annual reports (PDF, 10-K) plus quarterly financials (Excel/CSV). Any public company works; pick one and stick with it.

Handle **both** formats: `.xlsx` (structured tables) and `.pdf` (unstructured / semi-structured text and tables).

Convert the raw data into **usable chunks** suitable for retrieval and reasoning.

2. Understanding layer

Generate **"understanding files"** – derived artifacts (summaries, extracted metrics, metadata, embeddings/index, schema notes, or whatever your design needs) that let agents answer questions accurately instead of re-parsing raw files every time.

Be ready to explain what you chose to precompute vs. compute on the fly, and why.

3. Role-based access control (RBAC)

Different roles get different access. Enforce it **at the data layer**, not just in the UI.

Example rules (design your own role/permission model around these):

- **CEO** – access to all files and all data.
- **CTO** – access to most data, **but not Headcount / compensation data**.
- Add at least one more restricted role of your choosing.
The agent must **not leak restricted data** to a user who shouldn't see it – including when the answer would require combining restricted and permitted sources.

4. Feedback & learning loop

Implement a mechanism that lets the agents **learn from adoption** – i.e. improve as users interact with and correct the system.

This can be thumbs up/down, corrections, preferred answers, retrieval re-ranking, few-shot memory, eval-driven prompt updates – your call. Show that feedback actually changes future behavior.

5. Query interface

A user should be able to **ask questions in natural language**, and the agent(s) should fetch the relevant data and perform the next steps to answer.

The interface can be minimal (CLI, simple web UI, notebook) – we care that it works, not that it's pretty.

Bonus (not required, but rewarded)

Prompt injection handling — defend against malicious instructions embedded in the ingested documents or in user input.

Data

Use **public sources only**. Suggested: [Apple Investor Relations](https://investor.apple.com/) — annual reports (10-K), quarterly earnings, and financial statement Excel files. Any comparable public company is fine.

Do **not** use any real internal or confidential data.

Deliverables

- Working code** — a repository (GitHub or zip) that we can run. Include a short `README` with setup and run instructions, and note any keys/config we need to supply.
- A walkthrough** — either a recorded video (5–10 min) **or** a live walkthrough on the evaluation call. Cover: what you built, how the pieces fit, what you'd do differently with more time, and where it would break at scale.
- Any understanding files / indexes your system generates (or a script that regenerates them).

Submit the repo/zip link and walkthrough (or a note that you'll present live) by email within the 24- hour window.

Evaluation Criteria

Weight	Criterion	How it's judged
--------	-----------	-----------------

50%	**Working solution**	Does it run end-to-end? Ingest → understand → answer → enforce RBAC → feedback.
25%	**Architecture clarity & understanding**	Assessed during the walkthrough call — can you explain your design and defend your choices?
15%	**Ability to distinguish what fails at scale**	Assessed during the walkthrough — what breaks at 100× the data / users, and why?
5%	**Creativity**	Clever FE/BE hacks, elegant shortcuts, thoughtful UX.
5%	**Communication**	Clarity of README, walkthrough, and email questions.

A note on the split: half the score is simply **does it work.** Prioritize a running end-to-end system over breadth of features.

Ground Rules

Use any language, framework, model, or library you like.

AI coding tools are allowed — but you must be able to **explain every part** of what you submit during the walkthrough.

Scope aggressively. It's better to fully finish ingestion + RBAC + a simple feedback loop than to half-build everything.

Good luck — and get your questions to us within the first hour.